

Exercise 11

Arrays as Pointers

This exercise will give you a chance to work with arrays using the pointer syntax. You'll also get to use command-line arguments.

You'll find a source file, `sortList.c` and some sample inputs and expected output files on the course homepage in Moodle. If you're logged in on `remote.eos.ncsu.edu`, you can copy the files you need to the current directory using the following shell command:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise11/* .
```

This program is a partial implementation of a selection sort, using lots of pointer operations and pointer arithmetic. You'll need to fill in missing code in `main()` and a few other functions to get it working.

Once your program is working, you should be able to run it like the following example. Here, we're giving the name of an input file on the command line. The program reads a sequence of integers from the file, sorts them and then prints them out in sorted order, with a space after each value.

```
$ ./sortList input-1.txt  
74 166 250 273 441 545 659 710 808 879 924 931
```

Completing the Implementation

Fill in the missing parts of `main()`, `swap()`, `findSmallest()` and `printList()` functions to complete the program. For the `findSmallest()` and `printList()` functions, I've put constraints on what code you can add. This is to help make sure you get practice working with pointers. Instead of manipulating the array using the usual array syntax, we will access it via pointers to its elements.

As described in the source file, you get to complete four functions:

- **main()**

The main function already has code to create an array for storing the sequence of integers. It checks the command line arguments and prints out a usage message if the program isn't given the name of an input file to read. After this, you will need to open the input file and read a list of integers from it. If you can't open the file, your program should print the following message to standard error and then terminate with an exit status of 1. In this error message, *filename* should be the name of the file given on the command line.

```
Can't open file: filename
```

The `list[]` array only has room for 1000 integers. If the input file has too many numbers, your program should print the following message to standard error and then terminate with an exit status of 1.

```
Input list too long
```

- **swap()**

This is the usual function used to swap the contents of two integers passed in by address. You could probably find a copy of this function somewhere in the lecture notes or the example programs, but why not try to write it yourself, just for the practice.

- **findSmallest()**

Given an array (pointer to `int`) and a length, return a pointer to the smallest value in the array. In this function, you will use a pointer (rather than an index) to keep up with a particular element of an array.

- **printList()**

Given an array (pointer to int) and a length, print out the contents of the array. This function will let you traverse an array via a pointer (rather than an index).

For your output, print the sorted list of numbers on a single output line, with a space after each value. You don't have to worry about writing a fencepost loop; it's OK to have a space after the last output value.

Testing your Program

When your program is working, you should be able to run it like the example shown above and get a sorted list of numbers as output. If you want to make sure your program is producing exactly the right output, you can compare your output against the expected output file for that particular test case. For example, running your program like the following will capture output to a text file. After checking the program's exit status, we compare the program's output to the expected output for test case 1.

```
$ ./sortList input-1.txt > output.txt
$ echo $?
0
$ diff output.txt expected-1.txt
```

The **input-2.txt** input file contains more numbers, with multiple numbers on some input lines. You should be able to test with it using similar commands to test case 1:

```
$ ./sortList input-2.txt > output.txt
$ echo $?
0
$ diff output.txt expected-2.txt
```

The **input-3.txt** file is a test for error handling. It has more than 1000 numbers, so your program should print an error message to stderr and terminate unsuccessfully. The sample execution below shows capturing the standard error output to a file. After checking the program's exit status, we check the error message against an expected error message:

```
$ ./sortList input-3.txt 2> stderr.txt
$ echo $?
1
$ diff stderr.txt error-3.txt
```

The starter doesn't include an input-4.txt file, but it does have an error message for this test case. You can use this to check your program behavior when it can't open the given input file:

```
$ ./sortList input-4.txt 2> stderr.txt
$ echo $?
1
$ diff stderr.txt error-4.txt
```

Submitting your Solution

When you're done submit your completed sortList.c program via the exercise_11 assignment on Moodle.